



QuoteCat

Quoting for trades

QuoteCat — Project Flow, Beginning to End

Hey Mike,

You're not being a pain — you surfaced two real gaps, both of which we're fixing in the next release. Below is the flow as I see it should work today, plus the changes coming so it stops being confusing.

The path, at a glance

1. Create Quote
2. Send to Customer → Customer Approves (in their browser)
3. Convert to Contract
4. Sign as Contractor ← this step matters
5. Send to Customer → Customer Signs (in their browser)
6. Mark Complete
7. Create Invoice → Customer Pays
8. Mark Paid

Each step below tells you what to do, what the customer sees, and what changes in QuoteCat.

Step 1 — Create a Quote

In QuoteCat, tap the + on the Dashboard or open the Quotes tab and tap + **New Quote**.

Build your quote: client, project name, materials, labor, markup. The app shows you your **Materials Margin** and **Profit Margin** as you build. Watch the colored dots — green means you're at or above your target, yellow means close, red means you're underpriced.

When you're happy, save the quote.

Step 2 — Send the Quote to the Customer

Open the quote, tap **Review & Export**, then tap **Send to Client**. The app opens your share sheet — pick how you want to send (text, email, Messages, whatever). Your customer gets a link to view the quote in their browser.

The customer sees: A clean web page with your company branding, the line items, the total, and two buttons: **Accept** or **Decline**.

When the customer accepts, the quote's status updates to **Approved** in the cloud.

What's changing in v1.2.14: Today, your QuoteCat mobile app doesn't automatically refresh when you come back from your email/text app — you have to pull down to refresh. This was confusing because you'd approve the quote in the email, switch back to QuoteCat, and it would still show "Sent." We're fixing this so the app re-syncs automatically when you switch back to it.

Step 3 — Convert the Approved Quote to a Contract

From the approved quote, tap the **menu** and choose **Convert to Contract**.

The contract carries over the client info, scope, and totals. You can now edit terms, payment schedule, and conditions specific to the contract (separate from the quote pricing).

The new contract starts in **Draft** status.

Step 4 — Sign the Contract (as Contractor)

This is the step that tripped you up. You have to sign the contract before sending it to your client. The customer's web page won't let them sign until you've signed first.

In the contract editor, tap **Sign**. Draw your signature. Save.

What's changing in v1.2.14: The button on a draft contract used to say "Send to Client" whether or not you'd signed. We're changing it so it reads **"Sign"** until you've signed, then changes to **"Send"**. That way you can't accidentally send an unsigned contract — and you won't get stuck wondering why the customer's sign button doesn't work. After you sign, the app asks if you're ready to send it. Yes opens your share sheet so you can text or email the link. Tapping "No" leaves the contract in Draft — you can come back and send it whenever.

Step 5 — Send the Contract to the Customer

After you've signed, tap **Send**. The app confirms, then opens your share sheet to send the link. Status updates to **Sent**.

The customer sees: The contract on a web page with your signature already at the bottom. They have three buttons: **Sign**, **Request Changes**, or **Decline**.

Step 6 — What Happens Next

Three things the customer can do:

- **Sign** → contract status becomes **Signed**. Both signatures are recorded. You'll see a notification in QuoteCat.
- **Decline** → contract status becomes **Declined**. You'll see a notification. The conversation about why happens outside QuoteCat (call, text, email).
- **Request Changes** (NEW in v1.2.14) → status becomes **Changes Requested**, and the customer's feedback is attached to the contract. You'll see a notification and can read what they want changed.

If the customer requested changes, you'll need to **Revert to Draft** to edit. The app will prompt you: *"Reverting will clear both signatures. Continue?"* Tap yes, make your edits, sign again, and re-send.

Step 7 — Mark the Job Complete and Create the Invoice

Once you've actually done the work, open the signed contract and tap **Complete**. Status changes to **Completed**.

From there, tap **Create Invoice**. The invoice carries over the contract totals. Send it the same way you sent the quote and contract.

Step 8 — Get Paid, Mark Paid

When the customer pays, open the invoice and either log the payment manually or — if you're set up for card payments — the app will track it automatically. The invoice status updates to **Paid** or **Partial** depending on the amount.

Quick reference: contract button labels (after v1.2.14)

Contract status	The button you'll see	What it does
Draft, you haven't signed	Sign	Opens signature pad
Draft, you've signed	Send	Opens share sheet, status → Sent
Sent / Viewed	Share	Re-shares the link (status stays Sent)
Signed	Complete	Status → Completed
Completed	Export	Downloads the contract PDF

The status pill at the top is now read-only — it just tells you where the contract is. Forward motion happens via the button. The only way "backward" is the **Revert to Draft** action (which clears

signatures and restarts the signing flow).

Why your contract got stuck

For the record: when you sent the contract before signing it, the customer's web page correctly refused to let them sign — you can't have a signed contract where the contractor never agreed. But to you it looked like the sign button was broken. And once you tried to fix it by signing, the app wouldn't let you because the status was already "Sent" (we'd locked editing once it left Draft, which made sense in theory but trapped you in practice).

v1.2.14 fixes the upstream problem (you can't send without signing) AND the recovery path (when you do need to make changes, Revert to Draft is one tap and clear about what it does).

Thanks for the detailed feedback — this is exactly the kind of thing we needed to hear before launch. Let me know if any of this still doesn't match how you'd expect it to work.

— JoSeph